

JEGYZŐKÖNYV

Modern adatbázis rendszerek MSc

MongoDb C# APi

Készült: 2026. május 4.

Készítette: Kocsán László

Neptun kód: EK298H

Tartalomjegyzék

Bevezetés	2
1. MongoDB C# API	3
1.1. MongoClient	3
1.2. Database	3
1.3. Collection	3
1.4. CRUD műveletek	3
1.5. Model osztályok	4
1.6. Lekérdezések	4
1.7. Aggregáció	4
2. MongoDB API gyakorlati megvalósítása C# környezetben	5
2.1. Adatmodell kialakítása	5
2.2. Kapcsolódás és a kollekciók kezelése	5
2.3. Adatok lekérdezése	6
2.4. CRUD műveletek	7
2.5. Aggregációs műveletek	7
2.6. Kollektiók összekapcsolása	7
2.7. A megvalósítás értékelése	8
3. XDM modellből XML Schema készítése	9
3.1. Az XDM modell jellemzői	9
3.2. Transzformáció XDM modellből XML Schema-ba	9
3.3. Tervezési döntések	10
3.4. Összegzés	10
4. Saját XDM modell és XML Schema bemutatása	11
4.1. Az XDM modellem bemutatása	11
4.2. Az XDM modell XML Schema-ra konvertálása	11
4.2.1. Gyökérelem létrehozása	11
4.2.2. Entitások átalakítása komplex típusokká	11
4.2.3. Attribútumok használata azonosítókhoz	12
4.3. Kapcsolatok megvalósítása	12
4.3.1. 1:N kapcsolatok	12
4.3.2. Kulcsok és hivatkozások	12
4.3.3. Egyedi kulcs definiálása	13
4.3.4. Idegen kulcs kapcsolat	13
4.4. Összetett adatszerkezetek kialakítása	13
4.5. Adattípusok kiválasztása	13
4.6. Opcionális mezők kezelése	14
4.7. Összegzés	14

Bevezetés

Ez a dokumentum a Modern adatbázis rendszerek MSc kurzushoz kapcsolódó jegyzőkönyv, amely a félév során elvégzett feladatok és azok megvalósításának részletes bemutatását tartalmazza. A jegyzőkönyv célja, hogy átfogó képet adjon a modern adatbázis-technológiák gyakorlati alkalmazásáról, különös tekintettel a dokumentum-orientált és hierarchikus adatmodellekre.

A napjainkban egyre növekvő adatmennyiség és a különböző adatszerkezetek megjelenése szükségessé teszi a hagyományos relációs adatbázisokon túlmutató megoldások alkalmazását. Ennek megfelelően a jegyzőkönyvben bemutatott feladatok során a MongoDB dokumentumalapú adatbázis-kezelő, valamint az XML alapú adatmodellezés került a középpontba.

A dokumentum két fő részből épül fel:

- Az első rész a MongoDB C# API használatát mutatja be, beleértve a kapcsolatkezelést, a kollekciók működését, valamint az alapvető CRUD és aggregációs műveleteket.
- A második rész az XML alapú adatmodellezéshez kapcsolódik, ahol az XDM (XML Data Model) struktúrából történő XML Schema készítés kerül ismertetésre.

A jegyzőkönyv célja nem csupán az egyes technológiák bemutatása, hanem azok gyakorlati alkalmazásának szemléltetése is. A bemutatott példák egy vendéglátóipari adatmodellre épülnek, amely lehetővé teszi az összetettebb adatstruktúrák és kapcsolatok szemléltetését.

1. MongoDB C# API

A MongoDB C# API a MongoDB.Driver könyvtáron keresztül teszi lehetővé a MongoDB adatbázis kezelését .NET környezetben. Ez a driver biztosítja a kapcsolat létrehozását, a dokumentumok objektumokká alakítását, valamint a különféle adatkezelési műveletek végrehajtását.

1.1. MongoClient

A MongoDB szerverhez való kapcsolódásért a `MongoClient` objektum felelős:

```
1 var client = new MongoClient("mongodb+srv://...");
```

Ez a komponens biztosítja majd, a kapcsolatot az adatbázis-szerverrel, kezeli a kommunikációt, valamint optimalizálja a kapcsolatkezelést. A projekttem során MongoDB Atlas alapú felhőkapcsolatot alkalmaztam, amely lehetővé tette az adatoknak a távoli kezelését.

1.2. Database

Az adatbázisnak a kiválasztása a klienskapcsolaton keresztül történt:

```
1 var database = client.GetDatabase("vendeglatas");
```

A projekttemben a `vendeglatas` adatbázis került létrehozásra, amely a teljes rendszer logikai tárolóegységeként szolgált.

1.3. Collection

A kollekciók reprezentálják a dokumentumtároló egységeket:

```
1 var etteremCollection =  
2     database.GetCollection<Etterem>("ettermek");
```

A relációs adatbázisok tábláihoz hasonló szerepet töltenek be, ezek azonban sokkal rugalmasabbak. Egy kollekcióban található dokumentum nem feltétlenül rendelkezik teljesen azonos struktúrával. A generikus típusparaméter (`<Etterem>`) lehetővé tette, hogy a dokumentum automatikusan a megfelelő C# osztálypéldányá alakuljon.

1.4. CRUD műveletek

A C# API támogatja az Insert, Read, Update és Delete műveleteket. Ezek az adatbázis-kezelésnek az alapvető funkciói, amelyeknek a segítségével dokumentumokat lehet létrehozni, lekérdezni, módosítani és törölni. A feladatom során mind a négy műveletet alkalmaztam. Például új étteremnek a beszúrását, meglévő rekordnak a módosítását, illetve a törlést is megvalósítottam.

1.5. Model osztályok

A BSON dokumentumok C# osztályaihoz rendelhetők.

```
1 public class Etterem
2 {
3     [BsonId]
4     public ObjectId Id { get; set; }
5     public required string nev { get; set; }
6     public required Cim cim { get; set; }
7     public required int csillag { get; set; }
8 }
```

A modell osztályok a dokumentumoknak a szerkezetét definiálják.

A `[BsonId]` attribútum jelöli az elsődleges azonosítót, amelyet a MongoDB automatikusan fog kezelni. A `required` kulcsszó biztosítja, hogy az adott mező kötelezőnek kell lennie, ezáltal növelve az adatintegritását. A beágyazott objektumok mint például a `Cim` osztály lehetővé teszi az összetettebb dokumentumstruktúráknak a létrehozását.

1.6. Lekérdezések

```
1 var result = etteremCollection
2     .Find(e => e.csillag >= 4)
3     .ToList();
```

A lekérdezések lambda kifejezésekkel valósíthatók meg. A fenti példa minden olyan éttermet visszaad, amely legalább négycsillagos értékeléssel rendelkezik. A lekérdezések egyszerűen lehet kombinálni logikai operátorokkal.

1.7. Aggregáció

A MongoDB aggregációs pipeline-ja statisztikai és csoportosító műveleteket tesz lehetővé. Segítségével olyan műveletek hajthatók végre, mint:

- átlag kiszámítása
- darabszám meghatározása
- csoportosítás
- rendezés

A feladatom során például városenkénti átlagos csillagok számítását valósítottam meg. Az aggregációs műveletek különösen hasznosak nagyobb adatmennyiségek esetén, amikor statisztikai elemzésekre vagy üzleti riportok előállítására van szükség. A pipeline-nál a műveletek egymás után láncolva, jól strukturált módon hajtják végre az adatfeldolgozást.

2. MongoDB API gyakorlati megvalósítása C# környezetben

Ebben a fejezetben a feladatom gyakorlati részének részletes bemutatása következik. A fejlesztést Visual Studio környezetben, C# nyelven valósítottam meg, a MongoDB.Driver csomag felhasználásával. A program célja egy vendéglátóipari adatbázis létrehozása és kezelése volt, amelyen keresztül bemutathatóvá váltak a MongoDB API legfontosabb adatkezelési műveletei.

2.1. Adatmodell kialakítása

A feladatom egyik legfontosabb része az adatmodellnek a megtervezése volt. Mivel a MongoDB dokumentumalapú adatbázis, a relációs adatbázisokkal ellentétben itt nem táblákban és rekordokban, hanem objektumokként reprezentált dokumentumokban történik az adattárolás.

A rendszerben több különálló entitást hoztam létre:

- Az **Étterem** osztály az egyes vendéglátóhelyeknek az alapadatait tartalmazza, mint például a név, cím és csillag értékelés.
- A **Főszakács** és **Szakács** osztályok a dolgozói állományt reprezentálják. Ezekben tároltam a személyes adataikat, szakmai végzettségüket, valamint az étterem azonosítóit.
- A **Gyakornok** osztály külön figyelmet kapott, mivel itt már beágyazott objektumokat és tömböket is alkalmaztam. A gyakorlati időszaknak az adatai külön **Gyakorlat** objektumban szerepelnek, míg a műszakok listaként jelennek meg.
- A **Vendég** és **Rendelés** osztályok segítségével lehetővé vált a fogyasztási adatoknak a nyilvántartása.
- A **Cím** osztály beágyazott dokumentumként szerepel több entitásban is, ami jól bemutatja a MongoDB-nek az egyik legfontosabb előnyét.

Példa:

```
1 public required Cim cim { get; set; }
```

Ez azt jelenti, hogy a cím adatai nem külön kollekcióban, hanem közvetlenül a dokumentum részeként tárolódik.

2.2. Kapcsolódás és a kollekciók kezelése

Az adatbázissal való kapcsolat létrehozása a MongoClient objektum segítségével történt.

```
1 var client = new MongoClient(...);  
2 var database = client.GetDatabase("vendeglatas");
```

A `MongoClient` felelős a MongoDB szerverrel történő kommunikációért. A kapcsolat létrehozása után kiválasztottam a `vendeglatas` nevű adatbázist.

Ezt követően inicializáltam az egyes kollekciókat:

```
1 var ettermekCollection =  
2     database.GetCollection<Etterem>("ettermek");  
3  
4 var szakacsCollection =  
5     database.GetCollection<Szakacs>("szakacsok");
```

A kollekciók generikus típusparaméterrel történő létrehozása biztosítja a típus biztos működését. Ez azt jelenti, hogy a MongoDB dokumentumok automatikusan a megfelelő C# objektumokká fognak alakulni.

2.3. Adatok lekérdezése

A MongoDB C# API egyik legnagyobb előnye a könnyen olvasható, lambda-alapú lekérdezési szintaxis használ.

Teljes kollekció lekérdezése

```
1 var ettermek =  
2     ettermekCollection.Find(_ => true).ToList();
```

Ez a lekérdezés minden dokumentumot visszaad az adott kollekcióból. A `Find(_ => true)` feltétel minden rekordra igaz.

Feltételes szűrés

4 vagy több csillagos éttermek:

```
1 var result = ettermekCollection  
2     .Find(e => e.csillag >= 4)  
3     .ToList();
```

Ez a lekérdezés csak azokat az éttermeket adja vissza, amelyek legalább 4 csillagos értékeléssel rendelkeznek.

25 és 40 év közötti vendégek:

```
1 var result = vendegCollection  
2     .Find(v => v.eletkor >= 25 && v.eletkor <= 40)  
3     .ToList();
```

Itt már összetettebb logikai feltételt adtam meg.

2.4. CRUD műveletek

Beszúrás

```
1 etteremCollection.InsertOne(ujEtterem);
```

Ez egy új dokumentumot fog belehelyezni a MongoDB adatbázisba.

Módosítás

```
1 etteremCollection.UpdateOne(filter, update);
```

A művelet egy adott feltételnek megfelelő rekordot módosít. A feladatom során például egy étterem csillag értékelését módosítottam.

Törlés

```
1 foszakacsCollection.DeleteMany(filter);
```

A feladatomban a 30 év alatti főszakácsokat távolítottam el.

2.5. Aggregációs műveletek

Városonkénti átlag csillag értékelés:

```
1 var result = etteremCollection.Aggregate()  
2   .Group(e => e.cim.varos, g => new  
3     {  
4       Varos = g.Key,  
5       AtlagCsillag = g.Average(x => x.csillag)  
6     })  
7   .ToList();
```

A lekérdezés városok szerint csoportosítja az éttermeket, majd kiszámolja az adott város éttermeinek átlagos csillag értékelését.

2.6. Kollektiók összekapcsolása

A MongoDB-ben a relációs adatbázisok JOIN műveletéhez hasonló kapcsolatot a Lookup() függvény segítségével lehet megvalósítani.

```
1 var result = etteremCollection.Aggregate()  
2   .Lookup("szakacsok",  
3     "_ekod",  
4     "_e_sz",  
5     "szakacsok")  
6   .ToList();
```


A művelet során az éttermek kollekciót összekapcsoltam a szakácsok kollekcióval. A kapcsolatnak alapját az étterem azonosítója és a szakács dokumentumban tárolt hivatkozási mező adja. Ennek az az eredménye, hogy egyetlen lekérdezéssel megjeleníthetővé vált, hogy mely szakácsok tartoznak az adott étteremhez.

2.7. A megvalósítás értékelése

A gyakorlati feladat során sikeresen sikerült bemutatni a MongoDB C# API legfontosabb funkcióit, valamint megismerni a dokumentumorientált adatbázis-kezelés működését. A fejlesztés során lehetőség nyílt arra, hogy a MongoDB használatát ne csupán elméleti szinten, hanem valós programozási környezetben is alkalmazni tudjam. A vendéglátóipari adatbázis megfelelően megmutatja azokat az összetett kapcsolatokat, amelyek jól szemléltetik a dokumentumalapú adattárolás előnyeit. Különösen hasznosnak bizonyultak a beágyazott objektumok használata, mivel ezáltal az egymással szorosan összefüggő adatok természetes módon, egy dokumentumban tudtuk tárolni. Az aggregációs műveletek szintén jelentős szerepet kaptak, mivel ezek segítségével egyszerűen lehetett statisztikai jellegű lekérdezéseket végrehajtani. A fejlesztés során szerzett tapasztalatok alapján elmondható, hogy a MongoDB különösen alkalmas olyan alkalmazások fejlesztésére, ahol gyors fejlesztési ciklusra, rugalmas adatstruktúrára és összetett dokumentumkezelésre van szükség. Összességében a feladat hozzájárult ahhoz, hogy mélyebb gyakorlati ismereteket szerezzek a MongoDB és a C# alapú adatbázis-kezelés területén, valamint megmutatta, hogyan alkalmazható egy modern NoSQL adatbázis valós fejlesztési környezetben.

3. XDM modellből XML Schema készítése

A feladat célja az volt, hogy az XML adatmodell alapját képező XDM modellből egy formális, validációra alkalmas XML Schema hozzak létre. Az XDM modell az XML dokumentum logikai felépítését írja le fa struktúraként, ahol az adatok csomópontok (node-ok) formájában jelennek meg.

3.1. Az XDM modell jellemzői

Az XDM az XML dokumentumot egy hierarchikus fa struktúraként reprezentálja, amely az alábbi fő elemekből áll:

- Dokumentum csomópont (document node)
- Elem csomópontok (element nodes)
- Attribútumok (attribute nodes)
- Szöveges tartalom (text nodes)

Ez a modell lehetővé teszi az XML dokumentumnak az absztrakt leírását, azonban önmagában nem tartalmaz típusinformációkat vagy megszorításokat. Ennek biztosítására szolgál az XML Schema.

3.2. Transzformáció XDM modellből XML Schema-ba

A séma létrehozása során az XDM struktúrát leképeztem XSD elemekre. A főbb lépések a következők voltak:

- **Elemek definiálása:** Az XDM-ben szereplő element node-ok az XSD-ben `<xs:element>` formában jelennek meg.
- **Típusok meghatározása:** Az elemekhez egyszerű (`xs:string`, `xs:int`) vagy összetett típusokat (`xs:complexType`) rendeltem.
- **Hierarchia leképezése:** A beágyazott struktúrákat `<xs:sequence>` segítségével modelleztem.
- **Attribútumok kezelése:** Az XDM attribútum node-ok az XSD-ben `<xs:attribute>` formában kerültek definiálásra.
- **Kardinalitás megadása:** Az ismétlődő elemeknél a `minOccurs` és `maxOccurs` attribútumokat használtam.

3.3. Tervezési döntések

A séma kialakítása során több fontos döntést kellett meghoznom:

- **Attribútum vs. elem:** Az egyedi azonosítókat attribútumként definiáltam, míg a leíró adatokat elemekként.
- **Típusbiztonság:** Az XML Schema segítségével biztosítottam, hogy az adatok megfelelő típusúak legyenek.
- **Bővíthetőség:** A struktúrát úgy alakítottam ki, hogy később könnyen bővíthető legyen további elemekkel.

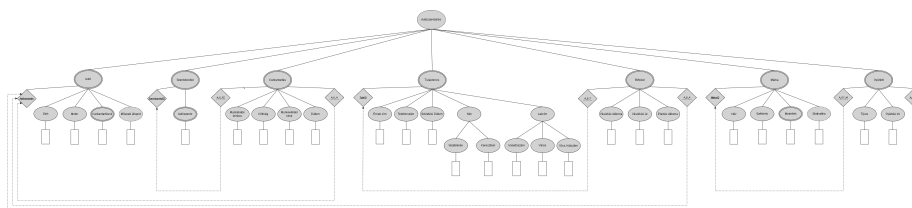
3.4. Összegzés

Az XDM modellből történő XML Schema készítés során sikerült az absztrakt, típus nélküli adatmodellt egy szigorúan validálható formába átültetni. Az XSD alkalmazása lehetővé tette az adatintegritás biztosítását, valamint a dokumentumok automatikus ellenőrzését különböző XML feldolgozó eszközök segítségével.

4. Saját XDM modell és XML Schema bemutatása

4.1. Az XDM modellem bemutatása

A rendszer tervezését először XDM diagram segítségével készítettem el. Az XDM modell az autószerzés fő entitásait és azoknak a kapcsolatait mutatja be. A modell központi eleme az **autoszervizeles**, amely az alábbi fő objektumokat tartalmazza:



1. ábra. Az autószerzés XDM modellje

Az XDM diagram alapján meghatározható volt:

- mely elemek lesznek önálló XML elemek,
- mely adatok legyenek attribútumok,
- milyen kapcsolat van az egyes entitások között,
- mely mezők kötelezőek,
- milyen adattípusokat kell használni.

4.2. Az XDM modell XML Schema-ra konvertálása

4.2.1. Gyökérelém létrehozása

Az XDM diagram legfelső eleme az **autoszervizeles** téma, ezért az XML Schema gyökérelme is ez lett:

```
1 <xs:element name="autoszervizeles">
```

Ebben helyezkednek el az összes további entitás.

4.2.2. Entitások átalakítása komplex típusokká

Az XDM modell minden fő entitását külön **complexType** típusként valósítottam meg. Például az **auto** entitásból létrejött az **autoTípus**:

```
1 <xs:complexType name="autoTípus">
```

Az elemhez tartoznak az alábbi tulajdonságok:

- szín
- motor
- muszakiallapot

amelyek, külön XML elemekként kerültek definiálásra.

4.2.3. Attribútumok használata azonosítókhoz

Az XDM diagramban szereplő azonosítókat attribútumként vagyis primary key-ként valósítottam meg.

```
1 <xs:attribute name="autorendszam"
2 type="xs:string"
3 use="required"/>
```

Ugyanezt alkalmaztam:

- tulajid
- markaid
- szervizbevitelid

esetén is.

4.3. Kapcsolatok megvalósítása

4.3.1. 1:N kapcsolatok

Az XDM modellben több helyen szerepeltek ismétlődő adatok. Ezeket az XML Schema-ban `maxOccurs="unbounded"` használatával valósítottam meg.

```
1 <xs:element name="auto"
2 type="autoTipus"
3 maxOccurs="unbounded"/>
```

Ez biztosítja azt, hogy több autó is szerepelhessen az XML dokumentumban.

4.3.2. Kulcsok és hivatkozások

Az XDM modellben kapcsolat van az egyes entitások között, például:

- karbantartás <-> autó
- birtokol <-> tulajdonos
- gyártott <-> marka

Ezeket az XML Schema-ban `xs:key` és `xs:keyref` segítségével valósítottam meg.

4.3.3. Egyedi kulcs definiálása

```
1 <xs:key name="auto_kulcs">
2   <xs:selector xpath="auto"/>
3   <xs:field xpath="@autorendszam"/>
4 </xs:key>
```

Ez biztosítja, hogy minden autónak a rendszáma egyedi legyen.

4.3.4. Idegen kulcs kapcsolat

```
1 <xs:keyref name="auto_karbantartas_fk"
2 refer="auto_kulcs">
```

Ez biztosítja azt, hogy a karbantartás csak létező autóra hivatkozhasson.

4.4. Összetett adatszerkezetek kialakítása

Az XDM modellben a **nev** és **lakcim** összetett szerkezetként szerepelt. Ezeket külön komplex típusokként hoztam létre.

```
1 <xs:complexType name="nevTípus">
```

A **nevTípus** tartalmazza:

- vezeteknev
- keresztnév

mezőket.

4.5. Adattípusok kiválasztása

Az XML Schema elkészítése során minden mezőhöz a megfelelő adattípust választottam.

Mező	Típus
datum	xs:date
koltseg	xs:decimal
gyartasiev	xs:integer
szin	xs:string

1. táblázat. Használt adattípusok

Az adattípusok használata biztosítja az XML dokumentum helyes validálását.

4.6. Opcionális mezők kezelése

Az XDM modell alapján az `eladasdatuma` mező nem minden esetben kötelező, ezért ezt opcionálisként definiáltam:

```
1 <xs:element name="eladasdatuma"  
2 type="xs:date"  
3 minOccurs="0"/>
```

Ez teszi lehetővé azt, hogy olyan autók is szerepeljenek az adatbázisban, amelyek még nem kerültek eladásra.

4.7. Összegzés

Az XDM modell alapján létrehozott XML Schema pontosan leképezi az autószervezelésnek a szerkezetét és kapcsolatait.

Az XSD biztosítja:

- az adatok egységes szerkezetét,
- az adattípusok ellenőrzését,
- a kapcsolatok validálását,
- valamint az XML dokumentumok konzisztenciáját.

A tervezés során külön figyelmet fordítottam arra, hogy a séma jól strukturált, könnyen bővíthető és logikusan felépített legyen.